

History of Java

- JAVA શરૂઆતમાં 1991 માં develop કરવામાં આવી ત્યારે તેને "OAK" નામ આપવામાં આવ્યું હતું, પરંતુ તેનું નામ બદલીને "JAVA" 1995 માં કરવામાં આવ્યું હતું.
- Java ને નાનું embedded systems device માટે બનાવવામાં આવી હતી જેવા કે સેટ-ટોપ બોક્સ.
- Platform-independent language બનાવી એ primary motivation હતું. જેનો use કરી ને કોઈપણ consumer electronic devices માં સોફ્ટવેર એમ્બેડે થાય છે.
- JAVA પ્રોગ્રામીંગ ભાષા મૂળ રીતે સન માઈક્રોસિસ્ટમ્સ દ્વારા વિકસાવવામાં આવી હતી જે જેમ્સ ગોર્સિલંગ દ્વારા શરૂ કરવામાં આવી હતી અને ૧૯૯૫ માં સન માઈક્રોસિસ્ટમ્સના કોર component તરીકે રજૂ કરવામાં આવી હતી. (Java 1.0 [J2SE]).
- Write Once, Run Anywhere (WORA), popular platforms પર no-cost run-times આપે છે.
- Java 2, નવા વર્ઝન વિવિધ પ્રકારનાં પ્લેટફોર્મ્સ માટે બનેલા છે. J2EE એ એન્ટરપ્રાઈઝ એપ્લિકેશન માટે technologies અને API નો સમાવેશ કરે છે. સામાન્ય રીતે server એન્વાયરમેન્ટ્સમાં ચાલે છે, જ્યારે J2ME API મોબાઈલ એપ્લિકેશન્સ માટે ઓપ્ટિમાઈઝ કરે છે.
- ડેસ્કટોપ version ને J2SE નું નામ આપવામાં આવ્યું હતું. 2006 માં, માર્કેટિંગ હેતુ માટે Sun Java EE, Java ME અને Java SE અનુક્રમે નવા J2 version નામ આપવામાં આવ્યું.
- 13 નવેમ્બર, 2006 ના રોજ, Sun એ General Public License (GPL) ની હેઠળ, મોટાભાગના JAVA ને ફ્રી અને open-source software (FOSS) તરીકે જાહેર કર્યું.
- 8 મે, 2007 ના રોજ, Sun એ Java ના બધા કોડને free અને open-source બનાવ્યાં જવા ના કોડનો એક નાનો ભાગમાં પણ Sun કૉપિરાઈટ ધરાવતો નથી.

Write and explain Features of JAVA. OR Explain advantages of JAVA.

- JAVA “Write Once, Run Anywhere” આપે છે, તે પ્લેટફોર્મ પર no-cost run-time પૂરું પાડે છે. ખૂબ secure અને configurable security આપે છે, તે નેટવર્ક અને ફાઈલ ઍક્સેસ ને allow કરે છે.

1) Simple (સિમ્પલ)

- ✓ તે સરળ છે કારણ કે તેમાં C અને C++ જેવી અન્ય language ના features હોય છે અને JAVA એ complexity ને દૂર કરે છે, કારણ કે તે પોઈન્ટર, સ્ટોરેજ Class અને goto statement નો ઉપયોગ કરતું નથી અને તે multiple Inheritance સપોર્ટ કરતું નથી.

2) Secure (સિક્યુઅર)

- ✓ જ્યારે code ને એક મશીનમાંથી બીજી મશીનમાં transfer કરીએ છીએ, ત્યારે તે પ્રથમ code ચેક કરે છે વાયરસ છે કે નહીં, તે કોડની safety ચેક કરે છે, જો તેમાં વાયરસ હોય તો તે કોડને ક્યારેય ચલાવશે નહીં.

3) Object-Oriented (ઑબ્જેક્ટ - ઑરિએન્ટેડ)

- ✓ આપણે જાણીએ છીએ કે pure object oriented language માં તમામ કોડ classes અને objects ના રૂપમાં છે.
- ✓ Java નો આ feature વધારે important છે. તે કોડ reusability અને maintainability ને સપોર્ટ કરે છે

4) Robust (રોબસ્ટ)

- ✓ પ્રોગ્રામ failures માટે બે મુખ્ય કારણો છે:
 1. Memory management mistake
 2. Mishandled exception or Run time errors
- ✓ JAVA ડાયરેક્ટ પોઈન્ટર મેનીપ્યુલેશનને સપોર્ટ કરતું નથી. આ JAVA પ્રોગ્રામને memory overwrite થવા દેતું નથી.
- ✓ જવા મેમરી allocation અને de-allocation પોતે manage કરે છે. de-allocation સંપૂર્ણપણે automatic છે, કારણ કે જવા unused object માટે garbage collection આપે છે.

- ✓ JAVA ઓબ્જેક્ટ-ઓરિએન્ટેડ exception handling આપે છે. સારી રીતે લખેલા JAVA પ્રોગ્રામમાં, તમામ run-time errors પ્રોગ્રામ દ્વારા manage કરી શકાય છે.

5) Multithreaded (મલ્ટિથ્રેડેડ)

- ✓ Thread એ એક સાથે અલગ અલગ પ્રોગ્રામ execute કરે છે.
- ✓ Multiple threads નો use કરી ને આપણે એવો JAVA પ્રોગ્રામ્સ લખી શકીએ છીએ જે ઘણીબધી tasks ને execute કરે છે.
- ✓ Multithreadingનો મુખ્ય ફાયદો એ છે કે તે same memory shares કરે છે.
- ✓ મલ્ટિમીડિયા અને વેબ એપ્લિકેશન માટે Threads મહત્વપૂર્ણ છે

6) Distributed (ડિસ્ટ્રિબ્યુટેડ)

- ✓ JAVAને ઇન્ટરનેટના distributed environment માટે ડિઝાઇન કરવામાં આવ્યું છે, કારણ કે તે TCP / IP protocols ને handle કરે છે
- ✓ JAVAમાં HTTP અને FTP જેવા વધારે ઉપયોગમાં લેવાતા protocol આવેલા છે.
- ✓ ઇન્ટરનેટ પ્રોગ્રામ્સ આ protocols use કરી ને કોલ કરી શકે છે અને ઇન્ટરનેટ પરની કોઈ પણ રીમોટ મશીનમાંથી ફાઇલોને access કરે છે.તેના માટે લોકલ સિસ્ટમ પર કોડ લખવાની જરૂર નથી.

7) Architecture-Neutral (આર્કિટેક્ચર- ન્યૂટ્રલ)

- ✓ તેનો અર્થ એ છે એક પ્લેટફોર્મમાં લખાયેલા પ્રોગ્રામ્સ ફરીથી લખી અથવા recompile કર્યા વગર કોઈપણ અન્ય પ્લેટફોર્મ પર ચાલી શકે છે. બીજા શબ્દોમાં કહીએ તો (write once, run anywhere, anytime, forever)
- ✓ JAVA પ્રોગ્રામ bytecode ફોર્મેટમાં compile થાય છે જે કોઈપણ મશીન depended નથી. પરંતુ તે મશીન માટે JVM દ્વારા machine code માં સરળતાથી translate કરી શકાય છે.
- ✓ જ્યારે applets અથવા applications developed કરવામાં આવે ત્યારે આ ખૂબ મદદરૂપ થશે જે કોઈપણ મશીન દ્વારા ડાઉનલોડ કરવામાં આવે છે અને કોઈપણ સિસ્ટમમાં ચાલે છે.

8) Platform Independent (પ્લેટફોર્મ ઇન્ડિપેન્ડન્ટ)

- ✓ તેનો મતલબ એ છે કે જ્યારે આપણે જાવામાં પ્રોગ્રામ compile કરીએ છીએ, ત્યારે તે પ્રોગ્રામનો byte code બનાવશે અને જ્યારે પ્રોગ્રામ રન કરીશું ત્યારે byte code execute થાશે.
- ✓ JAVAમાં તે ફરજિયાત નથી, કે જે ઓપરેટિંગ સિસ્ટમમાં JAVAમાં પ્રોગ્રામ બનાવે છે, તે જ ઓપરેટિંગ સિસ્ટમમાં આપણે પ્રોગ્રામને એક્ઝેક્યુટ કરવું પડશે.

9) Interpreted (ઈન્ટરપ્રિટેડ)

- ✓ મોટાભાગની programming languages ઓ compiled અથવા interpreted હોય છે, JAVA બંને compiled અને interpreted છે.
- ✓ Java compiler જાવા source ફાઇલને byte code માં translate કરે છે અને Java interpreter translate કરેલા byte code ને સીસ્ટમ પર સીધા જ execute કરે છે જે JVM દ્વારા કરવામાં આવે છે.

10) High Performance (હાય પેફોર્મન્સ)

- ✓ JAVA પ્રોગ્રામ્સને machine code ના બદલે byte code તરીકે ઓળખાતી intermediate representation માં convert કરવામાં આવે છે અને JVM કોઈપણ મશીન પર JAVA ચલાવે છે કે જેના પર JVM ઇન્સ્ટોલ કરેલું છે.
- ✓ JAVA bytecode ને ફક્ત Just-In-Time compile નો ઉપયોગ કરીને સારા performance માટે machine codeમાં translate કરવામાં આવે છે.
- ✓ તેથી, JAVA પ્રોગ્રામ્સ પ્રોગ્રામ અથવા સ્ક્રીપ્ટ્સ કરતાં વધુ ઝડપી છે જે સંપૂર્ણ રીતે interpreted languages માં લખાય છે પરંતુ C અને C++ પ્રોગ્રામ્સ કરતાં slow છે જે machine language માં compile થાય છે.

11) Dynamic (ડાઈનેમિક)

- ✓ Run time એ remote server પર રહેલા classes નો use કરી ને java એન્વાયર્નમેન્ટ extend થઈ શકે છે.
- ✓ Classes સાથે લિંક કરતી વખતે run time એ java interpreter name resolution કરે છે.
- ✓ Java interpreter મેમરીમાં ઓબ્જેક્ટની પ્લેસમેન્ટ નક્કી કરવા માટે જવાબદાર છે.

JDK (Java Development Kit)

- JAVA ડેવલપર ક્રિટ પ્રોગ્રામ્સ develop કરવા માટે ટૂલ્સ અને JRE આપે છે. JRE નો use કરી ને programs run થાય છે.
- JDK માં compiler (javac.exe), Java application launcher (java.exe) અને Appletviewer વગેરે જેવા tools આવેલા છે ... Compiler જાવા કોડને byte code માં translate કરે છે.
- Java application launcher JRE ને ઓપન કરે છે અને ક્લાસ લોડ કરી ને તેની main method ને call કરે છે.
- JAVA programs ચલાવવા માટે JRE ની જ જરૂર પડે છે. JRE java ફાઈલોને એક્ઝેક્યુશન કરવા માટે use થાય છે
- i.e. JRE = JVM + Java Packages Classes (like util, math, lang, awt, swing etc) + runtime libraries.

JRE (Java Runtime Environment)

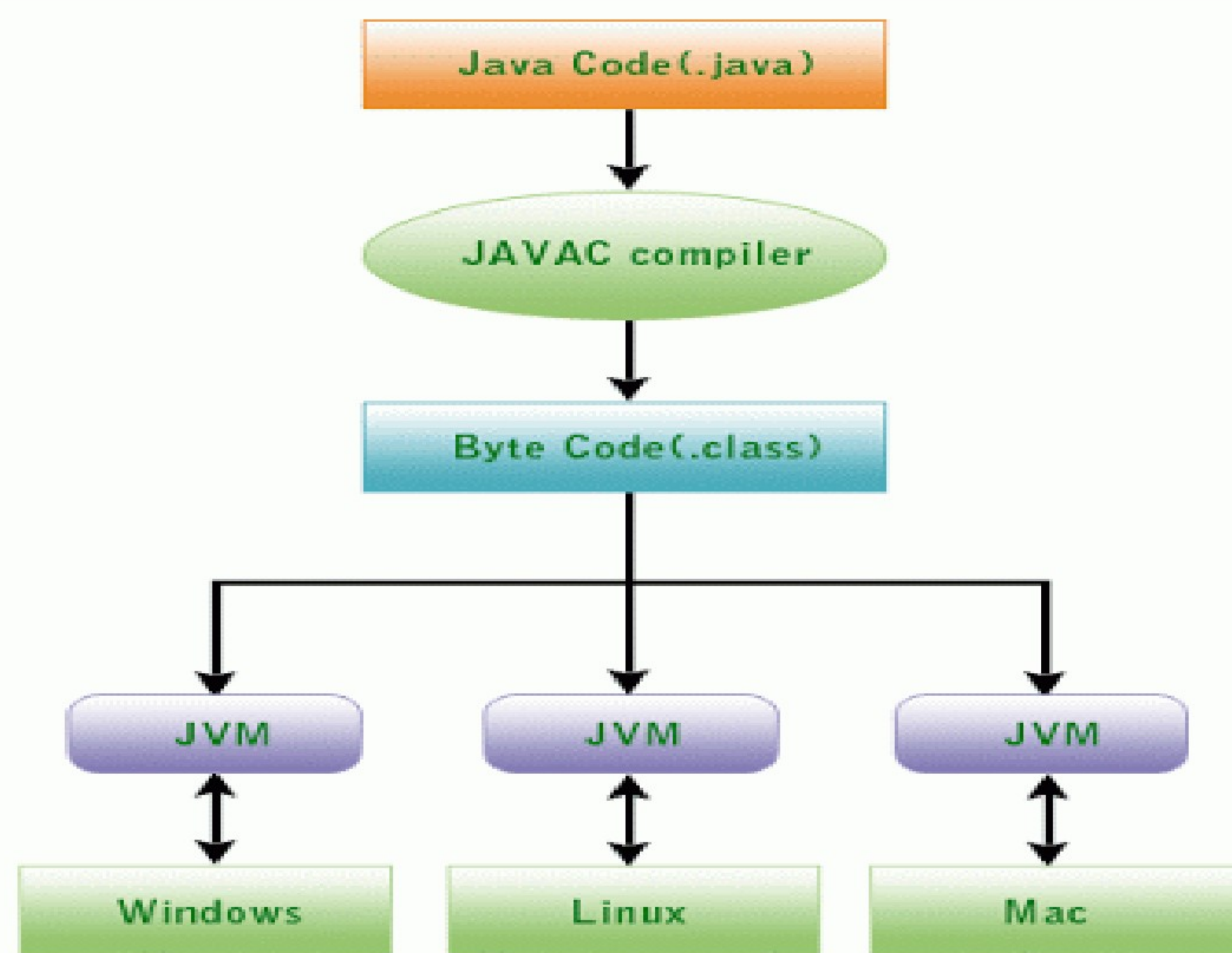
- Java Runtime Environment માં JVM, class libraries અને other supporting files હોય છે.
- તેમાં compiler, debugger વગેરે જેવા કોઈપણ development tools હોતા નથી.
- JVM પ્રોગ્રામ run કરે છે અને તે class libraries અને other supporting files નો ઉપયોગ કરે છે.

JVM (Java Virtual Machine)

- JVM ને Virtual કહેવામાં આવે છે કારણ કે તે મશીન ઈન્ટરફેસ પૂરું પાડે છે જે ઓપરેટિંગ સિસ્ટમ અને મશીન હાર્ડવેર આર્કિટેક્ચર પર આધારિત નથી.
- હાર્ડવેર અને ઓપરેટિંગ સિસ્ટમથી independent કરવાનો એક પાયો છે (WORA)
- જ્યારે આપણે જાવા ફાઈલ compile કરીએ છીએ ત્યાર આઉટપુટ '.exe' નથી પરંતુ તે '.class' ફાઈલ છે.
- '.Class' ફાઈલમાં byte codes છે જે JVM દ્વારા સમજી શકાય છે.
- JAVA Virtual મશીન bytecodeને મશીન કોડમાં ઓપરેટિંગ સિસ્ટમ અને હાર્ડવેર ના આધારે interprets કરે છે.
- Garbage collection અને array bounds checking વગેરે JVM દ્વારા થાય છે.
- JVM પોતે પ્લેટફોર્મ આધારિત છે.
- 2014 પછીના મોટાભાગના JVMs interpreter ને બદલે JIT (જસ્ટ ઈન ટાઈમ) compiler નો use કરે છે.

BYTE CODE

- Bytecode એ JAVA source code નું intermediate representation છે. જે source code ને compiler કરીને java compiler દ્વારા બનાવવામાં આવે છે.
- આ byte code એ મશીન independent code છે. તેને intermediate code પણ કહેવાય છે જે પછી JVM દ્વારા interpreted અને execute કરવામાં આવે છે.
- Bytecode JVM માટે એક મશીન કોડ છે. પરંતુ machine code એ platform specific છે જ્યારે byte code એ platform independent છે. જે તે વચ્ચેનો મુખ્ય તફાવત છે.
- તે .class ફાઈલમાં store છે જે source codeનું compiling કર્યા પછી બનાવવામાં આવે છે.



JAVA Environment Setup

- જ્યારે વિન્ડોઝ માટે path સેટ કરવો હોય ત્યારે તમે c: \ Program Files \ java \ jdk ડાયરેક્ટરીમાં જાવા ઈન્સ્ટોલ કર્યું છે.
- MY Computer પર રાઈટ-ક્લિક કરો અને 'propetry' પસંદ કરો.
- 'Advanced' ટેબ માં 'Environment variables' બટન પર ક્લિક કરો.
- ત્યારબાદ, System variables' માં 'Path' વેરિયેબલને બદલવું જેથી તેમાં Java executableનો પાથ હોય છે.
- ઉદાહરણ,જો અત્યારે path 'C:\WINDOWS\SYSTEM32' હોયતો, તેને change કરી ને 'C:\WINDOWS\SYSTEM32;c:\Program Files\java\jdk\bin' કરવો.

Describe the sample structure of JAVA program

- JAVA પ્રોગ્રામમાં ઘણા classes હોય છે કે જેમાં ફક્ત એક જ Class ને main method હોય છે
- Classesમાં data members અને methods હોય છે જે data members પર કાર્ય કરે છે.
- Methods માં data type declarations અને executable statements હોય છે.
- Java Programમાં નીચે બતાવ્યા પ્રમાણે એક અથવા વધુ sections આવેલ હોય છે.

Documentation Section	Suggested
Package Statement	Optional
Import Statements	Optional
Interface Statement	Optional
Class Definitions	Optional
Main Method class { Main Method Definition }	Essential

Documentation Section

- Documentation વિભાગમાં પ્રોગ્રામનું નામ, author અને અન્ય વિગતો comment linesમાં આપી શકાય છે, જે પ્રોગ્રામર પછી reference તરીકે use કરશે.
- Java પણ commentનો ઉપયોગ કરે છે જેમ કે / * * / ને documentation comment તરીકે ઓળખાય છે.
- આ ફોર્મનો documentation માટે ઉપયોગ થાય છે.

Package Statement

- Java ફાઈલમાં પ્રથમ statement એ package સ્ટેટમેન્ટ છે.
- આ statement પેકેજનું નામ declare કરે છે અને compilerને જણાવે છે કે અહીં defined કરેલા classes આ પેકેજના છે.

Example: Package Student;

Import Statements

- Package statement પછી એક કરતા વધારે import statements હોય શકે છે. આ c માં #include statement સમાન છે.

Example: Import student.test;

Interface Statement

- Interface એ class જેવું છે પરંતુ interface માં only method declarations હોય છે.
- Interface statement option છે અને તેનો ઉપયોગ જ્યારે આપણે પ્રોગ્રામમાં multiple inheritance implement કરવાનું હોય ત્યારે થાય છે.

Class Definition

- Java પ્રોગ્રામમાં multiple class definitions હોઈ શકે છે.
- JAVA પ્રોગ્રામમાં classes વધારે મહત્વ ધરાવે છે.
- આ classesનો ઉપયોગ real worldના ઓબ્જેક્ટ્સને map કરવા માટે થાય છે.

Main Method Class

- દરેક Java પ્રોગ્રામ માં main method હોવી જરૂરી છે. જે programની execution નો starting point છે.
- Simple java program માં ફક્ત આ એક part પણ હોય શકે છે.
- Main method વિવિધ classesની objects બનાવે છે અને તેમની વચ્ચે communication કરે છે.

Procedure-Oriented vs. Object-Oriented Programming

Procedure Oriented Programming (POP)	Object Oriented Programming (OOP)
• Procedures અને functions ને મહત્વ આપવામાં આવે છે.	• Procedures અને functions ને બદલે Dataને મહત્વ આપવામાં આવે છે.
• પ્રોગ્રામ ની ડિઝાઈન ટોપ ડાઉન હોય છે.	• પ્રોગ્રામ ની ડિઝાઈન બોટમ અપ હોય છે.
• મોટા programsને નાના programsમાં divide કરવામાં આવે છે જે ને functions કહે છે.	• મોટા programsને classes અને objectsમાં divide કરવામાં આવે છે.
• POPમાં કોઈ access specifier નથી.	• OOP માં Public, Private, Protected વગેરે નામના access specifier હોય છે.

મોટા ભાગના function શેરિંગ માટે Global ડેટાનો ઉપયોગ કરે છે જે system માં function થી function access થાય છે.	ડેટા function થી function access થતો નથી, તે public અથવા private રાખવામાં આવે છે જેથી ડેટાના ઍક્સેસને કન્ટ્રોલ કરી શકાય.
ડેટા અને function ને add કરવા મુશ્કેલ છે.	ડેટા અને function ને add કરવા સરળ છે.
Inheritance, polymorphism, data encapsulation, abstraction, access specifier જેવા Concepts નથી.	Inheritance, polymorphism, data encapsulation, abstraction, access specifier જેવા Concepts છે.
Examples: C, Fortran, Pascal, etc...	Examples: C++, Java, C#, etc...

Basic Concepts of OOP

- OOP ને વધુ powerful, secure, reliable અને સરળ બનાવવા માટે વિવિધ concepts આપે છે.

Object

- Object એ classનો instance છે.
- Real world માંથી object તરીકે person, computer વગેરે હોઈ શકે છે.
- દરેક objectમાં ઓછામાં ઓછી એક unique ઓળખ હોય છે.
- Object એ પ્રોગ્રામનો component છે જે program ના જુદા જુદા ભાગ સાથે interact કરે છે.
- object એ class type નો variable છે.

Class

- Class એ એક template છે જે objectsના attributes અને behaviorને દર્શાવે છે.
- Class એ blueprint અથવા prototype છે જેમાંથી objectsને બનાવવામાં આવે છે.
- Class એ objectsનો સમૂહ છે.
- Class એ abstract data type (ADT) નું implementation છે. એ attributes અને methods ને define કરે છે જે ADTના data structure અને operationsને implement કરે છે.

Data Abstraction

- background details નહિ ફક્ત essential features ને જ રજૂ કરે છે
- ડેટા ને secure કરવા માટે classમાં Implement કરવામાં આવે છે.
- Abstraction કરવા માટે abstract class અને interface ઉપયોગ કરીએ છીએ.

Encapsulation

- એક જ યુનિટમાં ડેટા અને functionsને Wrap up (બાઈન્ડીંગ) ને ઈન્કેપ્સ્યુલેશન તરીકે ઓળખવામાં આવે છે.
- Data બહારના world માટે ઍક્સેસિબલ નથી, માત્ર તે જ functions ઍક્સેસ કરી શકે છે જે class ની અંદર declare કરેલા હોય છે.

Inheritance

- Inheritance એ પ્રક્રિયા છે, જેના દ્વારા class અન્ય classની properties અને methods ને મેળવી શકે છે.
- જુના class માંથી નવા class ને બનાવવાની process ને inheritance તરીકે ઓળખવામાં આવે છે.
- નવા classને derived class કહેવાય છે અને જુના classને base class કહેવાય છે.
- Derived class માં base classના બધા features હોઈ છે અને પ્રોગ્રામર derived classમાં નવા features ઉમેરી શકે છે.

Polymorphism

- Polymorphism એટલે ability to take more than one form (એક કરતાં વધુ ફોર્મ લેવાની ક્ષમતા).
- તે એક કરતાં વધુ related purpose માટે એક નામનો ઉપયોગ કરી શકે છે.
- તેનો અર્થ એ થાય કે operator અને function એ different situations અલગ રીતે કાર્ય કરે છે.
- પોલીમોર્ફિઝમ કરવા માટે method overloading અને method overriding નો ઉપયોગ કરીએ છીએ.

Message Passing

- પ્રોગ્રામમાં ઓબ્જેક્ટનો સેટ હોય છે જે એકબીજા સાથે કોમ્યુનિકેશન કરે છે.
- કોમ્યુનિકેશન કરવા માટેના basic steps
 - Objects અને તેમના behavior defines કરવા માટે classes બનાવવા.
 - class ની definition માંથી objects બનાવવા.
 - objects વચ્ચે communication કરવું.

Dynamic and Static Binding

Static Binding	Dynamic Binding
objectનો પ્રકાર compile સમય (કમ્પાઈલર દ્વારા) પર નક્કી કરવામાં આવે છે, તે static binding તરીકે ઓળખાય છે.	objectનો પ્રકાર run-time પર નક્કી થાય છે, તે dynamic binding તરીકે ઓળખાય છે.
Static binding એ binding માટે Type માહિતીનો ઉપયોગ કરે છે.	Dynamic binding એ binding માટે Objectનો ઉપયોગ કરે છે.
static, private, final methods અને variables ને static binding ઉપયોગ કરીને resolve કરી શકાય છે.	રનટાઈમ દરમિયાન રનટાઈમ ઓબ્જેક્ટ નો use કરી ને Virtual methods ને resolve કરવામાં આવે છે.
Overloaded methods ને static bindingનો ઉપયોગ કરીને resolve કરી શકાય છે.	Overridden methods ને રનટાઈમ પર dynamic bindingનો ઉપયોગ કરીને resolve કરી શકાય છે.

Write a Program to print “My First Program In Java” in java.

```
/* Write a Program to print “My First Program In Java” in java.*/
import java.util.*;
class MyFirstProgram
{
    public static void main(String[] args)
    {
        // This statement will print the message.
        System.out.println("My First Program In Java");
    }
}
```